

EDA y Visualización

Interrogar los datos antes de modelarlos

Enrique Bolaños Abner Xocop

20 de agosto de 2026

Veritas · Libertas · Jvstitia

Agenda

- 1.** EDA: el interrogatorio
- 2.** Datos faltantes
- 3.** Outliers
- 4.** Distribuciones y relaciones
- 5.** Las herramientas
- 6.** Contar la historia
- 7.** Ejercicios
- 8.** Cierre

Mirar antes de modelar

El error más caro del curso pasado no fue elegir mal el modelo: fue entrenar sobre una columna que tenía 40 % de valores faltantes sin haberla mirado nunca.

El **EDA** (análisis exploratorio) no es hacer gráficas bonitas. Es un interrogatorio con una lista fija de preguntas:

1. **Forma:** ¿cuántas filas y columnas? ¿una fila es qué cosa?
2. **Tipos:** ¿numérica, categórica, fecha, texto?
3. **Faltantes:** ¿cuántos, dónde y por qué?
4. **Distribuciones:** ¿centro, dispersión, sesgo, extremos?
5. **Relaciones:** ¿qué se mueve junto con qué?

Ninguna de las cinco es opcional

El primer barrido

```
1 import pandas as pd
2 import seaborn as sns
3 df = sns.load_dataset("penguins")    # 344 filas
4 df.shape                            # (344, 7)
5 df.info()                          # tipos y no-nulos por columna
6 df.describe()                       # count, mean, std, min, cuartiles
7 df["species"].value_counts()
8 df.isna().sum()                     # faltantes por columna
```

Cinco líneas que se corren siempre, en ese orden, antes de escribir una sola línea de modelo. `describe()` revela mínimos imposibles (una edad de -1 , un precio de 0) y `value_counts()` revela las categorías escritas de tres formas distintas.

El mecanismo importa: MCAR, MAR, MNAR

Un NaN no es solo un hueco: es evidencia sobre *cómo* se generaron los datos.

- **MCAR** (completamente al azar): la falta no depende de nada. Se cayó el servidor un martes
- **MAR** (al azar dado lo observado): la falta depende de *otras* columnas que sí tenemos
- **MNAR** (no al azar): la falta depende del **valor que falta**

Solo bajo MCAR se puede borrar sin sesgar. Bajo MNAR, cualquier análisis que ignore el mecanismo está roto, y ningún truco de imputación lo salva.

La encuesta de ingresos

Preguntamos ingreso mensual y el 18 % no contesta.

- **MCAR:** se perdió un lote de formularios en la mudanza
- **MAR:** los jóvenes contestan menos, y sí tenemos la edad, así que podemos corregir usando la edad
- **MNAR:** los que ganan mucho no contestan *precisamente porque ganan mucho*

Si es MNAR e imputamos con la media, el ingreso promedio estimado sale **por debajo del real**, y lo reportaremos con total confianza.

MÁS ALLÁ DEL CÓDIGO

El mecanismo no se lee en los datos: se razona sobre cómo se recolectaron. Por eso el data scientist tiene que hablar con quien levantó la información.

Eliminar o imputar

```
1 df.isna().mean().round(3)      # fraccion faltante
2 # Marcar ANTES de tocar nada: la ausencia es señal
3 df["masa_faltaba"] = df["body_mass_g"].isna()
4 # Eliminar: solo si son pocos y sospechamos MCAR
5 df.dropna(subset=["sex"]).shape
6 # Imputar: la mediana aguanta mejor los extremos
7 m = df["body_mass_g"].median()
8 df["body_mass_g"] = df["body_mass_g"].fillna(m)
```

Spoiler de la próxima clase

Esa mediana se calcula **solo con los datos de entrenamiento**. Calcularla sobre todo el dataset es **leakage**, y el lunes veremos por qué eso infla los resultados y arruina el modelo.

Detectarlos: IQR y z-score

```
1 x = df["body_mass_g"].dropna()
2 q1, q3 = x.quantile(0.25), x.quantile(0.75)
3 iqr = q3 - q1
4 bajo, alto = q1 - 1.5*iqr, q3 + 1.5*iqr
5 x[(x < bajo) | (x > alto)]      # regla del boxplot
6 z = (x - x.mean()) / x.std()
7 x[z.abs() > 3]                 # regla z-score
```

El IQR usa cuartiles, así que los extremos no mueven el criterio. El z-score usa media y desviación, que **sí** los mueven: un outlier gigante infla std y termina escondiéndose a sí mismo. Con colas largas, prefieran IQR.

¿Error o señal?

Detectar un outlier es aritmética. Decidir qué hacer con él es criterio, y solo hay dos respuestas posibles:

- **Error:** una temperatura de 999, una edad de 400, un precio negativo. Se corrige o se marca como faltante
- **Señal:** el dato es real y raro. **Es probablemente lo más interesante del dataset**

En un análisis de retail, el cliente que compraba 200 veces más que el resto fue borrado por ser *outlier*. Era el mayorista que sostenía el 30 % de la facturación.

Fraude, fallas y fugas viven en la cola de la distribución

La forma de una variable

Gráfica	Muestra	Cuidado
Histograma	la forma completa	el <i>bin</i> cambia la historia
Boxplot	mediana, IQR, extremos	esconde la bimodalidad
Violin	forma + resumen	confunde a audiencias no técnicas

Un boxplot de dos poblaciones mezcladas (dos jorobas) se ve idéntico al de una sola población ancha. Por eso el histograma va **primero**: es el único que no promedia la forma antes de enseñárnosla.

Prueben siempre dos o tres cantidades de bins antes de concluir algo.

Colas largas y el log

Muchas variables económicas (ingreso, precio, ventas, población) tienen cola derecha larga: pocos valores enormes arrastran todo.

```
1 import numpy as np
2 precios = np.array([100, 120, 150, 90, 8000.0])
3 precios.mean()      # 1692: lo arrastro el extremo
4 np.median(precios)  # 120: resiste
5 np.log1p(precios)   # comprime la cola derecha
```

La transformación logarítmica vuelve simétrica la distribución, hace visible la estructura del grueso de los datos y convierte relaciones multiplicativas en aditivas. Usen `log1p` cuando haya ceros.

Correlación: el número que esconde la forma

El coeficiente de Pearson r mide **asociación lineal**, nada más:

- $r = 0$ no significa independencia: una parábola perfecta da $r \approx 0$
- r alto no significa causalidad (lo veremos en serio el 29 de septiembre)
- r es sensible a outliers: un punto puede fabricarlo o destruirlo

El **cuarteto de Anscombe** son cuatro datasets con la misma media, la misma varianza y el mismo $r = 0,816$, y cuatro formas completamente distintas. El *Datasaurus* lleva el chiste al extremo: doce datasets con estadísticos idénticos, uno de ellos un dinosaurio.

Nunca reporten un r sin haber visto el scatter

Matplotlib: siempre fig y ax

```
1 import matplotlib.pyplot as plt
2 fig, ax = plt.subplots(figsize=(6, 4))
3 ax.scatter(df["bill_length_mm"], df["body_mass_g"])
4 ax.set_xlabel("Largo del pico (mm)")
5 ax.set_ylabel("Masa (g)")
6 ax.set_title("Picos largos, cuerpos pesados")
7 fig.tight_layout()
```

Eviten el estilo `plt.plot()` suelto: funciona en el notebook y se rompe apenas necesiten dos paneles. Con `fig`, `ax` tienen objetos explícitos, y `plt.subplots(1, 2)` da un arreglo de ejes.

Seaborn: EDA en una línea

```
1 sns.histplot(df, x="body_mass_g", hue="species")
2 sns.boxplot(df, x="species", y="body_mass_g")
3 sns.scatterplot(df, x="bill_length_mm",
4                  y="body_mass_g", hue="species")
5 sns.pairplot(df, hue="species") # todo contra todo
6 num = df.select_dtypes("number")
7 sns.heatmap(num.corr(), annot=True, cmap="vlag")
```

Seaborn entiende DataFrames y colorea por categoría sin esfuerzo: es la herramienta para **explorar**. Matplotlib es la herramienta para **pulir** lo que ya decidieron mostrar. Plotly existe si necesitan interactividad (zoom, tooltips) en un dashboard.

¿Qué gráfica uso?

La pregunta decide la gráfica, no al revés:

Pregunta	Gráfica	Evitar
¿Cuál es mayor?	barras	pie con 8 rebanadas
¿Cómo se distribuye?	histograma, box	barra del promedio
¿Se mueven juntas?	scatter	barras apiladas
¿Cómo evoluciona?	línea	barras por año
¿Qué parte del todo?	barra apilada, treemap	pie 3D

Regla práctica: si tienen que explicar cómo leer la gráfica antes de explicar el hallazgo, elijan otra gráfica.

Storytelling: la gráfica afirma algo

- **Título que afirma**, no que describe. “Ventas por región” no dice nada; “Occidente cayó 22 % mientras el resto creció” sí
- **Anoten el punto clave** en la gráfica misma (una flecha, un texto). Si el lector tiene que buscarlo, no lo va a encontrar
- **Quiten tinta inútil**: rejillas pesadas, bordes, fondos, leyendas redundantes, decimales que nadie usa
- **Una gráfica, una idea**. Dos ideas son dos gráficas

MÁS ALLÁ DEL CÓDIGO

En el Proyecto 1 evaluamos hallazgos, no cantidad de gráficas. Veinte gráficas sin conclusión valen menos que tres que responden la pregunta.

Ética: cómo mienten las gráficas

Todas estas son técnicamente correctas y todas engañan:

- **Eje truncado:** empezar el eje y en 98 para que una diferencia de 0.5 % parezca un derrumbe
- **Ventana escogida:** mostrar los 3 meses que suben dentro de una serie de 5 años que baja
- **Doble eje:** dos escalas distintas ajustadas hasta que las curvas “coincidan”
- **Áreas mal escaladas:** duplicar el ancho y el alto de un ícono cuadruplica el área para representar el doble
- **Eje invertido:** el truco de la gráfica de homicidios de Florida

Si la gráfica necesita el truco, el hallazgo no existe

Ejercicio 1: EDA guiado

Corran el checklist completo sobre penguins y escriban **tres hallazgos** en texto, no en gráficas.

```
1 import seaborn as sns
2 df = sns.load_dataset("penguins")
3 # a) forma, tipos y faltantes
4 # ¿Que va aqui?
5 # b) distribucion de body_mass_g por species
6 # c) la relacion mas fuerte entre dos numericas
7 # d) escriban 3 hallazgos en una celda markdown
```

Ejercicio 2: clasificar los faltantes

```
1 df.isna().sum()
2 # a) ¿que columnas tienen faltantes y cuantos?
3 # b) ¿las filas con sex faltante son distintas del
4 #     resto en masa o en especie? (comparen medias)
5 # c) argumenten: ¿MCAR, MAR o MNAR? Justifiquen
6 # d) propongan una estrategia y digan que sesgo
7 #     introduce
```

No hay respuesta única. Lo que se evalúa es el argumento.

Ejercicio 3: la gráfica mentirosa

Construyan la versión tramposa y la honesta del mismo dato.

```
1 ventas = [100.0, 101.2, 100.8, 102.0, 101.5]
2 anos = [2021, 2022, 2023, 2024, 2025]
3 # a) version mentirosa: eje y de 100 a 102
4 # ¿Que va aqui?
5 # b) version honesta: eje y desde 0
6 # c) ¿que titulo pondria cada uno?
```

Resolvemos los tres en el notebook de la sesión; lo que falte queda de práctica para la casa.

Lo esencial de hoy

- EDA es un **checklist**: forma, tipos, faltantes, distribuciones, relaciones
- El **mecanismo** del faltante (MCAR, MAR, MNAR) decide qué se puede hacer con él
- Un outlier es **error o señal**: hay que decidir cuál, no borrarlo por reflejo
- Colas largas: mediana e IQR, y el log como aliado
- **Nunca un r sin su scatter** (Anscombe, Datasaurus)
- Seaborn para explorar, Matplotlib para pulir, título que afirma

Próxima clase (martes 25): Feature Engineering

Recuerden: la **HDT 3** se entrega ese mismo martes